



Olist-Ecommerce-Analytics

E-COMMERCE BUSINESS INTELLIGENCE & CUSTOMER EXPERIENCE ANALYTICS PLATFORM

Capstone Project Report

Python / Pandas

Streamlit

Plotly

Scikit-Learn

Analyst

Brojo Mohan Dutta

Period

September 2016 – October 2018

Submitted

2026

AICTE | IBM SkillsBuild Data Analytics with AI Internship 2026

Field	Detail
Project Title	Olist-Ecommerce-Analytics
Domain	E-Commerce Analytics / Business Intelligence / Data Analytics / Machine Learning
Dataset	Olist Brazilian E-Commerce Public Dataset
Author	Brojo Mohan Dutta
Education	B.Sc. (Honours) Mathematics, Serampore College, University of Calcutta, 2025
Repository	https://github.com/brojomohan58-boop/Olist-Ecommerce-Analytics
Submission Context	AICTE IBM SkillsBuild Data Analytics with AI Internship 2026

Table of Contents

1. Executive Summary
2. Introduction
3. Project Background
4. Problem Statement
5. Project Objectives
6. Dataset Description
7. Data Architecture
8. Data Preparation and Processing
9. Technology Stack
10. Application Architecture
11. Dashboard Modules
12. Business Questions Addressed
13. Machine Learning – Delay Predictor
14. Interactive Filtering
15. Analytical Methodology
16. Validation and Testing
17. Project Structure
18. Installation and Execution
19. GitHub Repository
20. Key Insights and Business Value
21. Limitations
22. Future Improvements
23. Project Outcome
24. Conclusion
25. Author Information

1. Executive Summary

This report documents the design, engineering, and validation of an end-to-end **Business Intelligence and Customer Experience Analytics Platform** built on the **Olist Brazilian E-Commerce Public Dataset**. The platform converts nine raw, relationally linked marketplace tables — customers, orders, order items, products, sellers, payments, reviews, and geolocation — into a governed analytical layer capable of supporting operational and strategic business decisions.

The delivered artifact is a production-style, multi-page **Streamlit** application exposing eight analytical modules: Executive Overview, Products & Categories, Delivery & Operations, Customer Experience, Payments, Regional Analysis, Seller Intelligence, and a machine-learning-driven **Delay Predictor**. Each module is powered by interactive **Plotly** visualizations and a **centralized filtering engine** that guarantees cross-page metric consistency.

The engineering approach followed six disciplined stages: (1) systematic data auditing, (2) cleaning and type correction, (3) construction of two purpose-built analytical grains, (4) KPI design and validation, (5) structured exploratory data analysis (EDA), and (6) leakage-controlled predictive modeling. A dedicated regression-test suite of **eleven validation scripts** was executed across the data pipeline and every dashboard module prior to submission, returning **zero failures** across all reported checks (see Section 16).

Analytical integrity was treated as a first-class constraint throughout the build: sales and freight values are never represented as profit in the absence of Cost of Goods Sold (COGS) data; statistical associations (e.g., delivery delay vs. review score) are never asserted as causal; and machine-learning outputs are surfaced strictly as **probability estimates**, never as deterministic guarantees. This discipline is what elevates the project from a technical demonstration to a **defensible, decision-support-grade analytics platform**, suitable for real operational use by a marketplace business.

The result is a reproducible, version-controlled, fully tested analytics application that demonstrates applied proficiency across the full data analyst lifecycle: data engineering, data modeling, KPI governance, exploratory analysis, BI dashboard development, applied machine learning, and software validation.

2. Introduction

Modern e-commerce marketplaces generate high-volume, high-dimensional transactional data spanning multiple interdependent domains: customer acquisition and geography, product catalog structure, order fulfillment and logistics, payment processing, and post-purchase satisfaction signals. The business value locked inside this data is rarely accessible through raw tables alone — it requires a structured analytical translation layer that connects granular records to well-defined business questions, governed KPIs, and interpretable, decision-ready insights.

This project was undertaken as the capstone deliverable of the **AICTE | IBM SkillsBuild Data Analytics with AI Internship 2026**, a program that explicitly emphasizes the responsible use of AI-assisted development tooling — in this case, **IBM Bob** — in combination with sound, independently verifiable analytical judgment. A central premise of the internship curriculum is that AI-assisted tools accelerate implementation velocity but do not, and cannot, substitute for the analyst's accountability over correct business logic, rigorous data validation, and honest interpretation of statistical results. This report reflects that principle throughout: every analytical claim made by the platform is scoped to what the underlying data can actually support.

The **Olist Brazilian E-Commerce Public Dataset** was selected as the analytical foundation for this capstone because it offers a realistic, multi-table view of marketplace operations — order lifecycle timestamps, item-level transaction detail, seller and customer geography, granular payment structures, and customer review scores — providing sufficient breadth and relational complexity to justify a genuine, multi-module BI platform rather than a single-table dashboard exercise.

3. Project Background

The Olist dataset contains anonymized order records from a real Brazilian e-commerce marketplace that connects small and medium-sized sellers to major online sales channels. The data spans orders placed between late 2016 and late 2018, structured across **nine relational tables** describing customers, orders, order items, products, sellers, payments, reviews, geolocation, and a Portuguese-to-English product category translation table.

Because this dataset reflects genuine production marketplace operations rather than a synthetic, single-purpose teaching sample, it exhibits characteristics typical of real-world operational data:

- **Inconsistent completeness across tables** — some fields (e.g., product dimensions, review comment text) are populated inconsistently.
- **Unresolved order statuses** — not every order resolves to a final delivered state within the observed window; some remain shipped, canceled, unavailable, or processing.
- **A partial initial period** — the final quarter of 2016 shows very low order volume relative to 2017–2018, indicating platform ramp-up rather than a mature, representative operating baseline.
- **Structural field gaps** — no cost-of-goods-sold (COGS) field exists anywhere in the schema, which permanently constrains any attempt at true profitability analysis.

This background directly shaped the analytical approach adopted in this project. Before any dashboard or model was constructed, every one of the nine source tables was audited individually — row counts, null distributions, duplicate keys, and referential integrity across joins were all inspected — and the analytical scope of the platform was deliberately and explicitly narrowed to what the dataset could honestly support. Where the data could not support a claim (e.g., profit, guaranteed causation), the platform does not make that claim.

4. Problem Statement

Marketplace operators modeled on the Olist structure require integrated visibility into how **sales performance, delivery reliability, and customer satisfaction** interact across product categories, geographic regions, and individual sellers. This visibility does not exist by default within raw relational transaction data: order, item, payment, and review records are distributed across separate tables at different natural levels of granularity (order-level vs. item-level), making it structurally difficult — and analytically risky — to answer even basic operational questions without a purpose-built analytical layer that correctly respects those grains.

This project addresses that gap directly. It delivers a unified, validated analytics platform capable of answering, with defensible rigor:

- **Where** does the business generate sales, and how has that evolved over time?
- **How reliably** are orders delivered relative to their estimated delivery date?

- **How does delivery performance relate to** customer satisfaction — understood as an association, not a proven cause?
- **How do customers pay**, and how does installment behavior vary?
- **Which regions and sellers** drive marketplace activity and scale?
- **Can delivery-delay risk be estimated** before an order's final outcome is known, using only information available at or near order placement?

Every one of these questions is answered without overstating what the available data can prove — a constraint that governs every downstream design decision in this report.

5. Project Objectives

The project was scoped and executed against the following nine objectives:

1. **Audit and clean** the nine raw Olist datasets into analysis-ready, type-correct, deduplicated form.
2. **Establish a data architecture** appropriate to the two distinct natural grains present in the data — order-level and order-item-level — rather than forcing a single flattened table.
3. **Define business-relevant KPIs** that can be correctly and consistently calculated from the fields actually available in the schema.
4. **Conduct structured exploratory data analysis (EDA)** to surface observations, insights, and testable hypotheses across the sales, delivery, and customer-experience domains.
5. **Develop a delivery-delay risk prediction component** using strictly leakage-safe features and probability-based outputs.
6. **Build an interactive, multi-page Streamlit application** that presents the full analysis to a business-stakeholder audience.
7. **Apply centralized, consistent filtering** across all eight dashboard pages, eliminating the risk of cross-page metric drift.
8. **Validate the correctness** of the data pipeline, every KPI calculation, every chart, and overall application behavior through structured, automated test scripts.
9. **Document the platform's methodology, architecture, and limitations transparently**, so that evaluators and future maintainers understand exactly what the platform can and cannot claim.

6. Dataset Description

The project uses the **Olist Brazilian E-Commerce Public Dataset**, composed of the following nine source files:

#	File	Description
1	olist_customers_dataset.csv	Customer identifiers and location (city, state, zip code prefix)
2	olist_geolocation_dataset.csv	Latitude/longitude reference data indexed by zip code prefix
3	olist_order_items_dataset.csv	Line-item detail per order: product, seller, price, freight value
4	olist_order_payments_dataset.csv	Payment type, number of installments, and payment value per order

#	File	Description
5	olist_order_reviews_dataset.csv	Customer review score and review timestamps per order
6	olist_orders_dataset.csv	Order status and full lifecycle timestamps
7	olist_products_dataset.csv	Product category and physical dimension attributes
8	olist_sellers_dataset.csv	Seller identifiers and seller location
9	product_category_name_translation.csv	Mapping from Portuguese to English product category names

These nine tables were joined, cleaned, and transformed into the analytical datasets described in Section 7. Each source table was profiled independently prior to any join operation, confirming primary-key uniqueness where applicable (e.g., `order_id` in `olist_orders_dataset.csv`) and cardinality on foreign-key relationships (e.g., the one-to-many relationship between orders and order_items).

7. Data Architecture

A central architectural decision in this project is the deliberate use of **two distinct analytical grains** rather than a single flattened, denormalized table. This decision underpins the correctness of every KPI in the platform.

7.1 Order-Level Dataset — `master_df`

- **Grain:** One row per order
- **Validated shape:** 99,441 rows × 42 columns
- **Used for:** order-count KPIs, sales and order-level metrics, delivery analysis, customer experience, review analysis, payment analysis, regional analysis, and order-status analysis

7.2 Order-Item-Level Dataset — `order_item_master`

- **Grain:** One row per order item
- **Validated shape:** 112,650 rows × 35 columns
- **Used for:** product analysis, category analysis, seller analysis, item-level sales, item quantity, freight analysis, and product/category trend analysis

7.3 Why Two Grains Are Structurally Necessary

A single order can legitimately contain multiple order items — potentially spanning multiple products and, in some cases, multiple sellers. If item-level records are merged directly into an order-level table without prior aggregation, every order-level field becomes duplicated once per item, silently inflating any metric that should represent a single order exactly once.

Concretely: an order containing three items would appear three times in a naively flattened join, causing order-count-based KPIs — such as Total Orders or Average Order Value — to be systematically overstated. The inverse failure mode is equally damaging: collapsing all item-level detail into an order-level aggregate discards the granularity required for product- and seller-level analysis, such as category-level sales comparisons or seller performance ranking.

By maintaining `master_df` strictly at the order grain and `order_item_master` strictly at the item grain, every analytical question in the platform is routed to the dataset whose granularity matches the question:

```
Order-level question -> master_df      (e.g., total orders, AOV, delivery time)
Item-level question  -> order_item_master (e.g., category sales, seller ranking)
```

This design decision is enforced programmatically throughout `src/kpis.py` and `src/charts.py`, and is directly covered by the automated KPI validation suite described in Section 16.

7.4 Entity Relationship Overview

```
customers --1:N-- orders --1:N-- order_items --N:1-- products --N:1-- category_translation
      |                                     |
      |                                     +--N:1-- sellers --1:N-- geolocation (seller zip)
      |
      +--1:N-- order_payments
      +--1:1-- order_reviews

customers --N:1-- geolocation (customer zip)
```

8. Data Preparation and Processing

The data preparation workflow followed a strict, auditable sequence:

10. **Data loading** — all nine raw CSV files loaded and row/column counts verified against expected reference values.
11. **Data cleaning** — datetime parsing and type correction, deduplication, and explicit column-by-column handling of missing values.
12. **Data transformation** — category name translation, geolocation deduplication, and text-field standardization.
13. **Derived fields** — delivery time in days, delivery delay in days, and the binary delay target (`is_delayed`), calculated only for orders with a valid delivered status and a non-null actual delivery date.
14. **Analytical dataset construction** — assembly of the two analytical grains described in Section 7, plus a dedicated, leakage-safe machine-learning dataset.
15. **Post-transformation validation** — row counts, null distributions, and target class balance re-checked after every transformation step.
16. **Full-suite validation** — KPI validation, chart validation, application validation, and regression testing, detailed in Section 16.

The final analytical datasets produced by this pipeline are:

- `data/olist_master_cleaned.csv`
- `data/olist_order_item_master.csv`
- `data/ml_delay_dataset.csv`

8.1 Default Dashboard Filter Configuration

To guarantee consistent and comparable analysis across the platform, the following default filters are applied on every dashboard page at application startup:

Filter	Default Value
Date Start	2017-01-01
Date End	2018-10-17
Order Status	Delivered
Include 2016	OFF

These defaults deliberately exclude the low-volume, non-representative partial period of 2016, and restrict analysis by default to orders that reached a resolved delivered status, ensuring that delivery and customer-experience metrics are calculated on a consistent, resolved population. These values are treated as validated defaults and are not altered without explicit, documented justification.

8.2 Data Cleaning and Merge Pipeline (Python / pandas)

The following script demonstrates the core cleaning logic applied to the raw Olist tables, including datetime parsing, geolocation and review null handling, and construction of the item-level analytical grain.

```
"""
data_pipeline.py
Cleans and merges the raw Olist tables into the two analytical grains
used throughout the BI platform: master_df (order-level) and
order_item_master (item-level).
"""

import pandas as pd
import numpy as np

RAW_PATH = "raw/"
OUT_PATH = "data/"

# 1. LOAD RAW TABLES
customers = pd.read_csv(f"{RAW_PATH}olist_customers_dataset.csv")
orders = pd.read_csv(f"{RAW_PATH}olist_orders_dataset.csv")
order_items = pd.read_csv(f"{RAW_PATH}olist_order_items_dataset.csv")
payments = pd.read_csv(f"{RAW_PATH}olist_order_payments_dataset.csv")
reviews = pd.read_csv(f"{RAW_PATH}olist_order_reviews_dataset.csv")
products = pd.read_csv(f"{RAW_PATH}olist_products_dataset.csv")
sellers = pd.read_csv(f"{RAW_PATH}olist_sellers_dataset.csv")
geoloc = pd.read_csv(f"{RAW_PATH}olist_geolocation_dataset.csv")
cat_trans = pd.read_csv(f"{RAW_PATH}product_category_name_translation.csv")

# 2. DATETIME PARSING (orders table)
date_cols = [
    "order_purchase_timestamp", "order_approved_at",
    "order_delivered_carrier_date", "order_delivered_customer_date",
    "order_estimated_delivery_date",
]
for col in date_cols:
    orders[col] = pd.to_datetime(orders[col], errors="coerce")

reviews["review_creation_date"] = pd.to_datetime(reviews["review_creation_date"], errors="coerce")
reviews["review_answer_timestamp"] = pd.to_datetime(reviews["review_answer_timestamp"],
errors="coerce")
```

```

# 3. GEOLOCATION CLEANING - collapse multiple lat/long rows per zip
geoloc_clean = (
    geoloc.groupby("geolocation_zip_code_prefix", as_index=False)
    .agg(geolocation_lat=("geolocation_lat", "median"),
         geolocation_lng=("geolocation_lng", "median"),
         geolocation_city=("geolocation_city", "first"),
         geolocation_state=("geolocation_state", "first"))
)
geoloc_clean["geolocation_state"] = geoloc_clean["geolocation_state"].str.upper()

# 4. REVIEWS CLEANING - handle missing comment text, dedupe by order
reviews["review_comment_title"] = reviews["review_comment_title"].fillna("")
reviews["review_comment_message"] = reviews["review_comment_message"].fillna("")
reviews_clean = (
    reviews.sort_values("review_answer_timestamp")
    .drop_duplicates(subset="order_id", keep="last")
)

# 5. CATEGORY TRANSLATION
products = products.merge(cat_trans, on="product_category_name", how="left")
products["product_category_name_english"] =
products["product_category_name_english"].fillna("unknown_category")

# 6. CUSTOMER / SELLER GEOGRAPHY STANDARDIZATION
customers["customer_state"] = customers["customer_state"].str.upper()
customers["customer_city"] = customers["customer_city"].str.title()
sellers["seller_state"] = sellers["seller_state"].str.upper()
sellers["seller_city"] = sellers["seller_city"].str.title()

# 7. DERIVED DELIVERY FIELDS - only for resolved, delivered orders
delivered_mask = (
    (orders["order_status"] == "delivered")
    & orders["order_delivered_customer_date"].notna()
)

orders["delivery_time_days"] = np.where(
    delivered_mask,
    (orders["order_delivered_customer_date"] - orders["order_purchase_timestamp"]).dt.days,
    np.nan,
)
orders["delivery_delay_days"] = np.where(
    delivered_mask,
    (orders["order_delivered_customer_date"] - orders["order_estimated_delivery_date"]).dt.days,
    np.nan,
)
orders["is_delayed"] = np.where(
    delivered_mask,
    (orders["order_delivered_customer_date"] >
orders["order_estimated_delivery_date"]).astype(int),
    np.nan,
)

# 8. ORDER-LEVEL GRAIN - master_df (one row per order)
master_df = (
    orders
    .merge(customers, on="customer_id", how="left")
    .merge(reviews_clean, on="order_id", how="left")
    .merge(
        payments.groupby("order_id", as_index=False).agg(
            payment_value=("payment_value", "sum"),
            payment_installments=("payment_installments", "max"),
            payment_type=("payment_type", "first")),
        on="order_id", how="left")
)

```

```

        .merge(
            geoloc_clean.rename(columns={"geolocation_zip_code_prefix": "customer_zip_code_prefix"}),
            on="customer_zip_code_prefix", how="left")
    )

# 9. ITEM-LEVEL GRAIN - order_item_master (one row per order item)
order_item_master = (
    order_items
    .merge(orders[["order_id", "order_status", "order_purchase_timestamp", "customer_id"]],
           on="order_id", how="left")
    .merge(products, on="product_id", how="left")
    .merge(sellers, on="seller_id", how="left")
    .merge(customers[["customer_id", "customer_state", "customer_city"]],
           on="customer_id", how="left")
)

# 10. VALIDATION CHECKPOINT
assert master_df.shape[0] == orders["order_id"].nunique(), "master_df grain violated"
assert order_item_master.shape[0] == order_items.shape[0], "item grain violated"

print(f"master_df:          {master_df.shape}")
print(f"order_item_master: {order_item_master.shape}")

# 11. PERSIST ANALYTICAL DATASETS
master_df.to_csv(f"{OUT_PATH}olist_master_cleaned.csv", index=False)
order_item_master.to_csv(f"{OUT_PATH}olist_order_item_master.csv", index=False)

```

This pipeline enforces the two-grain architecture programmatically (Section 7), applies null-handling decisions explicitly rather than implicitly, and produces the exact validated shapes reported in Sections 7.1 and 7.2.

9. Technology Stack

Technology	Role in the Project
Python	Core programming language for data processing, analysis, and application logic
Pandas	Data loading, cleaning, transformation, aggregation, and construction of the two analytical grains
NumPy	Numerical operations and vectorized calculations, including derived delivery and delay fields
Plotly	Interactive chart rendering within the Streamlit dashboard pages
Streamlit	Web application framework used to build the multi-page BI dashboard interface
Scikit-learn	Machine learning model training and probability prediction for the delivery-delay predictor
Git	Version control for source code and application history
GitHub	Remote repository hosting, project publication, and submission reference
Git LFS	Large File Storage, used to track the analytical CSV datasets within the GitHub repository
PowerShell	Command-line environment used for local environment setup and application execution on Windows

10. Application Architecture

The platform is implemented as a **Streamlit multi-page application** with a strict separation between the presentation layer (pages) and the analytical logic layer (source modules). This separation is not cosmetic — it is what allows the centralized filtering system (Section 14) and the validation suite (Section 16) to operate deterministically against shared, single-source-of-truth logic rather than per-page duplicated code.

Main application entry point: app.py

Dashboard pages:

- 17. pages/01_overview.py
- 18. pages/02_products_categories.py
- 19. pages/03_delivery_operations.py
- 20. pages/04_customer_experience.py
- 21. pages/05_payments.py
- 22. pages/06_regional.py
- 23. pages/07_sellers.py
- 24. pages/08_ml_delay_predictor.py

Core analytical modules:

Module	Responsibility
src/data_loader.py	Loads the cleaned analytical datasets and prepares them for downstream use, with caching
src/filters.py	Implements the centralized filtering system applied consistently across all dashboard pages
src/kpis.py	Calculates the validated business KPIs used throughout the dashboards
src/charts.py	Builds the Plotly visualizations used across all dashboard pages
src/ml_module.py	Contains the delivery-delay prediction model logic, feature preparation, and probability output

10.1 Sample Application Entry Point and Centralized Filter Routing

```
"""
app.py
Streamlit multipage entry point. Loads cleaned datasets once (cached),
initializes the centralized filter state, and renders the landing page.
"""

import streamlit as st
from src.data_loader import load_master_data, load_item_data
from src.filters import init_filter_state, render_filter_sidebar, apply_filters

st.set_page_config(
    page_title="Olist E-Commerce BI Platform",
    page_icon="\U0001F4E6",
    layout="wide",
```

```

    initial_sidebar_state="expanded",
)

# 1. LOAD DATA ONCE - cached across all pages in the session
@st.cache_data(show_spinner="Loading analytical datasets...")
def load_all_data():
    master_df = load_master_data("data/olist_master_cleaned.csv")
    item_df = load_item_data("data/olist_order_item_master.csv")
    return master_df, item_df

master_df, item_df = load_all_data()

# 2. INITIALIZE CENTRALIZED FILTER STATE (session-wide, cross-page)
init_filter_state(
    default_start="2017-01-01",
    default_end="2018-10-17",
    default_status="delivered",
    include_2016=False,
)

render_filter_sidebar()

# 3. APPLY CENTRALIZED FILTERS - identical call signature on every page
filtered_master = apply_filters(master_df, grain="order")
filtered_items = apply_filters(item_df, grain="item")

st.session_state["filtered_master"] = filtered_master
st.session_state["filtered_items"] = filtered_items

# 4. LANDING CONTENT
st.title("E-Commerce Business Intelligence Platform")
st.markdown(
    """
    Use the sidebar to navigate between analytical modules:
    Overview, Products & Categories, Delivery & Operations,
    Customer Experience, Payments, Regional, Sellers, Delay Predictor.

    All pages share the same filtered population, defined in the sidebar,
    guaranteeing consistent cross-page metrics.
    """
)
st.metric("Orders in current filter", f"{filtered_master.shape[0]:,}")

```

Each dashboard page under pages/ imports `st.session_state['filtered_master'] / st.session_state['filtered_items']` rather than re-implementing its own filter logic — enforcing the centralized filtering principle described in Section 14 at the code level, not merely as a design intention.

11. Dashboard Modules

11.1 Section 1 — Executive Overview

The Executive Overview page provides a high-level summary of overall business performance, including sales performance, order volume, core business KPIs, and time-based trends. It is designed to give a management-level audience a quick, consolidated view of how the business is performing over the selected filter period, without requiring navigation into the more detailed operational pages.

Key KPIs presented: Total Sales Value, Total Orders, Average Order Value (AOV), Month-over-Month Growth, Delivered vs. Non-Delivered Order Split.

Analytics

Executive Overview

Products & Categories

Delivery & Operations

Customer Experience

Payments

Regional Analysis

Seller Intelligence

Machine Learning

Delay Predictor

Info

About

Filters

Analysis Period

From

To

2017/01/01

2018/10/17



Executive Overview

High-level KPIs, monthly order & revenue trends, and order status breakdown. Data grain: one row per order. Default view: delivered orders, Jan 2017 – Oct 2018.

Key Performance Indicators

Total Orders

96,211

Total Sales Value

R\$ 13,181,027

Total Freight

R\$ 2,192,093

Avg Order Value

R\$ 137.00

Median Order Value

R\$ 86.50

Unique Customers

93,104

Cancellation Rate

0.60%

On-Time Delivery

91.9%

Delay Rate

8.1%

Avg Delivery Days

12.1 days

Median Delivery

10.0 days

Avg Delay (late)

8.9 days

Avg Review Score

4.16 / 5

5-Star Rate

59.2%

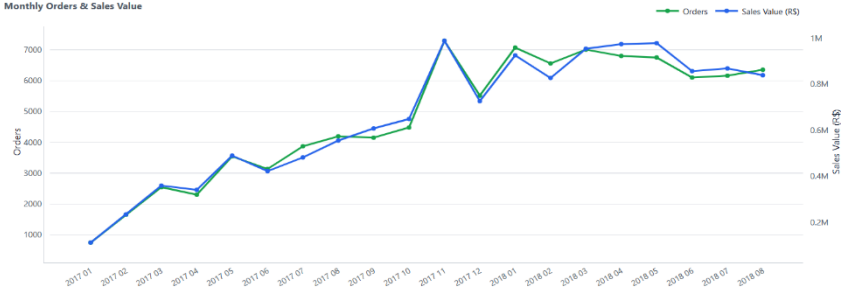
1-2 Star Rate

12.8%

Delivery KPIs: 96,203 orders with resolved delivery date. Review KPIs: 95,568 orders with a review score (99.3% coverage). Avg Delay shows late orders (is_delayed = 1) only.

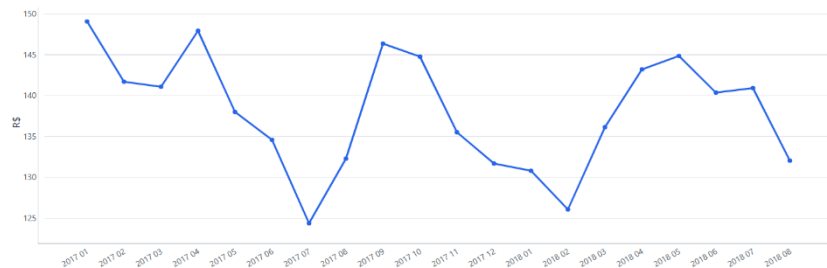
Monthly Order Volume & Sales Value

Monthly Orders & Sales Value



Monthly Average Order Value

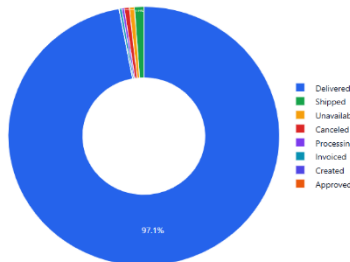
Monthly Average Order Value (R\$)



Order Status Distribution

Shows all order statuses in the selected date range, regardless of the status filter applied to delivery and value KPIs above.

Order Status Share

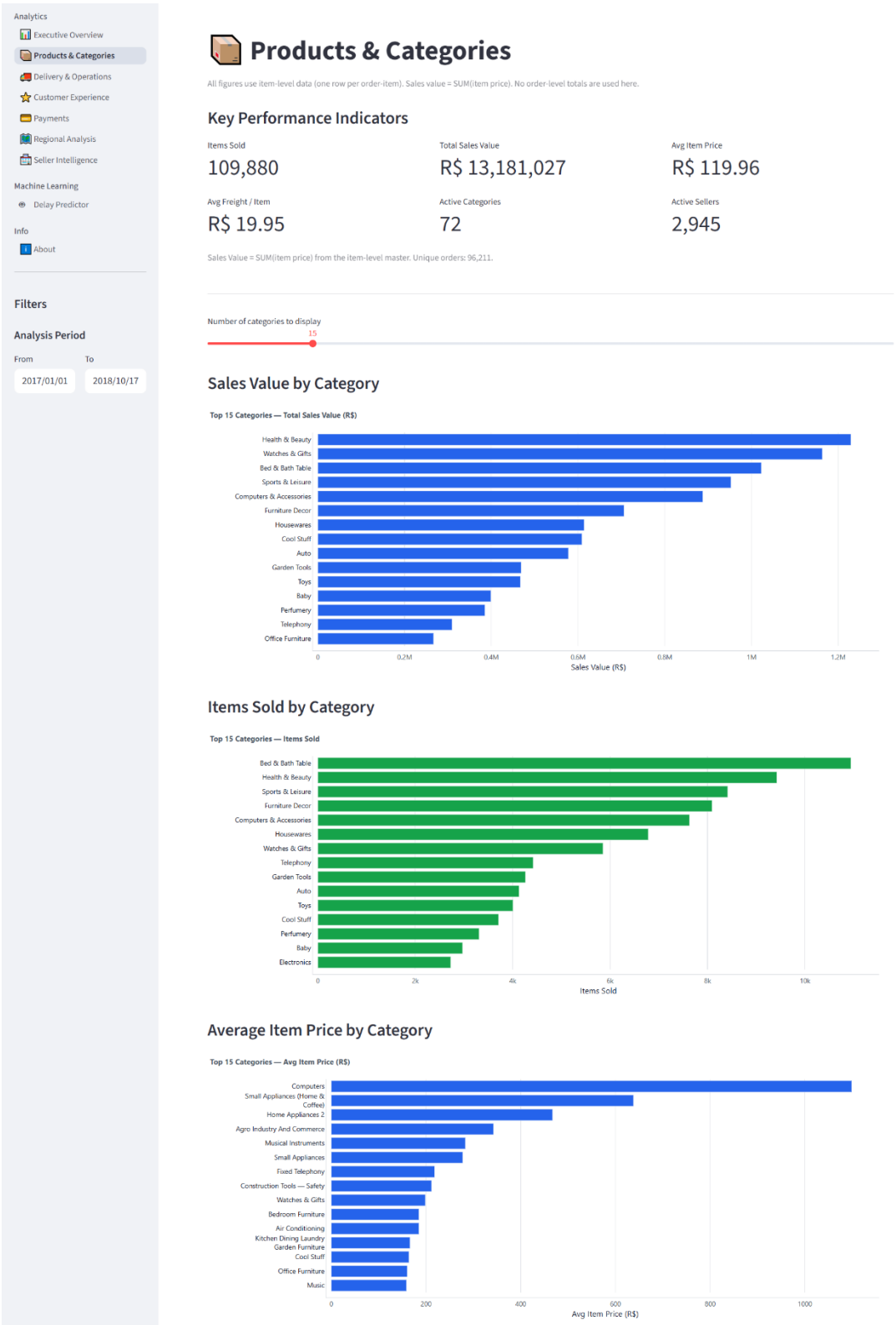


Order counts by status

Status	Orders	Share
delivered	96211	97.07%
shipped	1098	1.11%
unavailable	602	0.61%
canceled	599	0.60%
processing	299	0.30%
invoiced	296	0.30%
created	5	0.01%
approved	2	0.00%

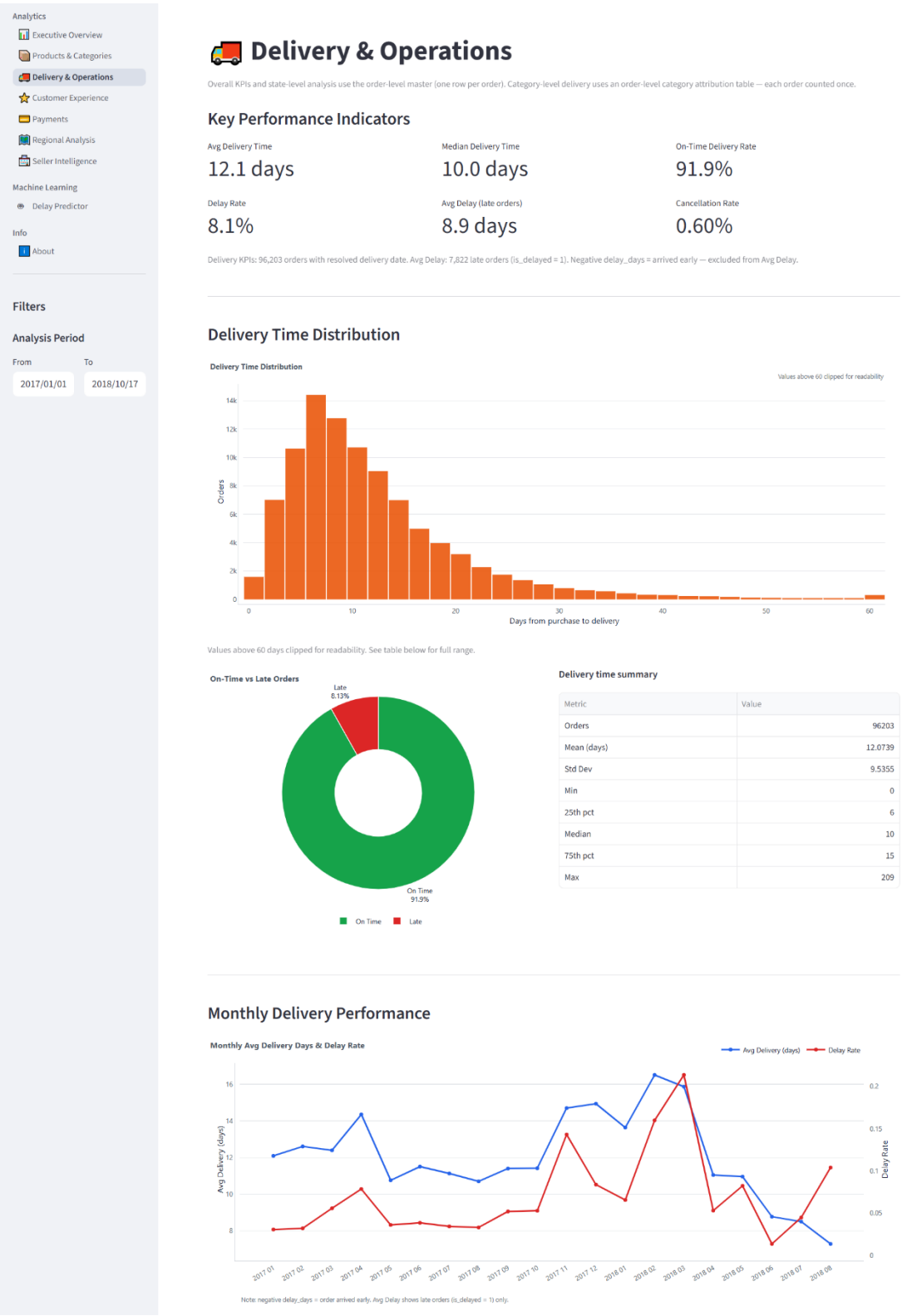
11.2 Section 2 — Products & Categories

The Products & Categories page operates on the item-level analytical grain (order_item_master) and presents sales value by category, items sold, average item price, average freight value, monthly item-sales trends, and a comparison of category price against volume, along with category-level detail views.



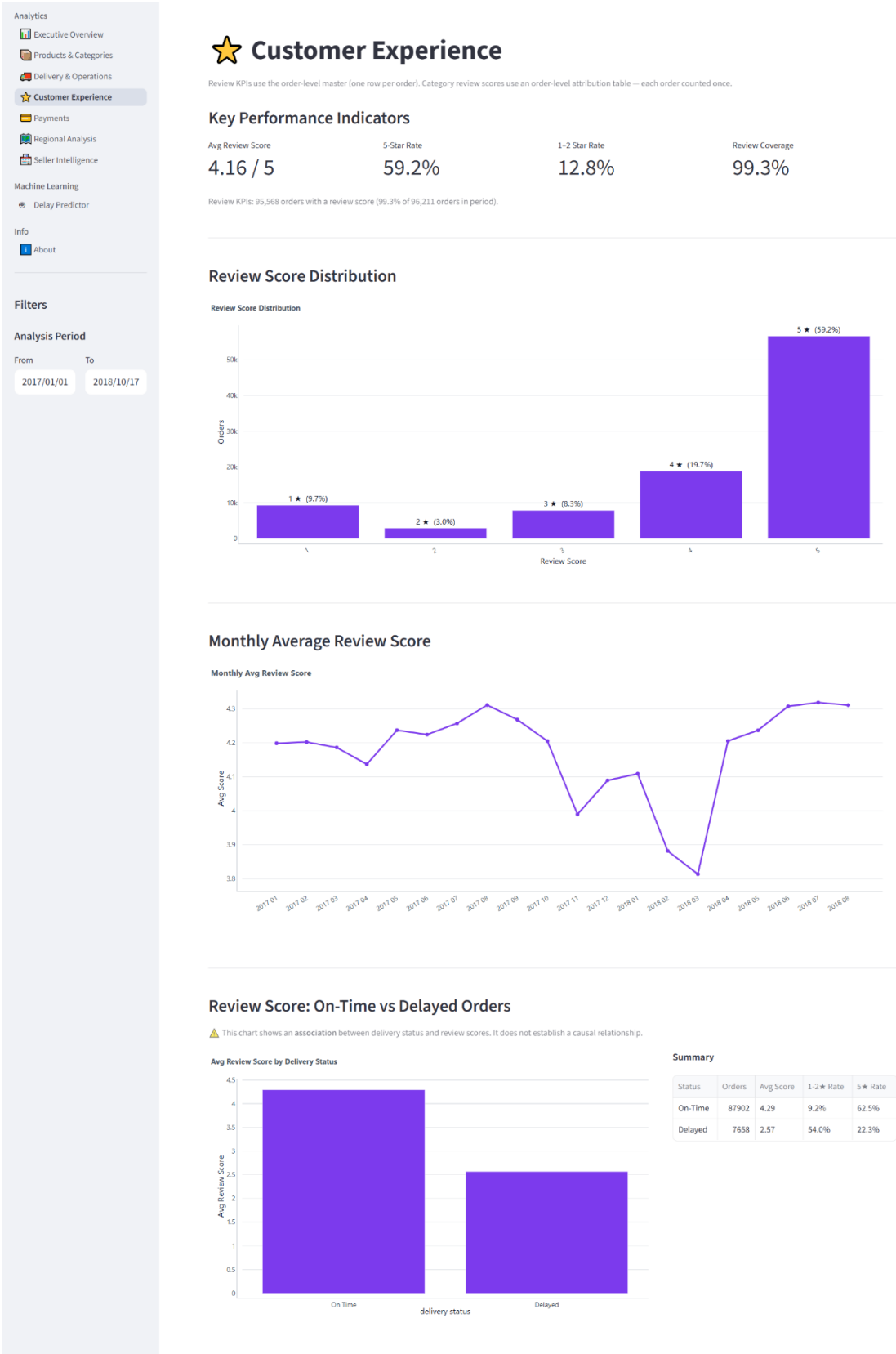
11.3 Section 3 — Delivery & Operations

The Delivery & Operations page analyzes delivery time, the relationship between estimated and actual delivery dates, delayed-order rates, delivery performance trends over time, and related operational KPIs. Delivery performance is presented strictly as a factor that may relate to customer experience; the platform does not claim delivery performance is the sole or proven cause of customer-experience outcomes.



11.4 Section 4 — Customer Experience

The Customer Experience page analyzes review-score distributions, customer-experience patterns, and the relationship between delivery outcomes and customer-experience metrics.



11.5 Section 5 — Payments

The Payments page analyzes payment methods, payment values, installment behavior, payment-value distributions, and payment trends over time.

Analytics

Executive Overview

Products & Categories

Delivery & Operations

Customer Experience

Payments

Regional Analysis

Seller Intelligence

Machine Learning

Delay Predictor

Info

About

Filters

Analysis Period

From

To

2017/01/01

2018/10/17

Payments

All payment metrics use the order-level master (one row per order). Payment vs-sales difference is a reconciliation metric only -- no causal attribution.

Key Performance Indicators

Credit Card Share

75.7%

Avg Order Value

R\$ 137.00

Avg Installments (CC)

3.52

High-Installment (>6)

16.0%

Payment type share: 99,112 orders (all statuses). Installment KPIs: 75,015 credit-card orders.

Payment Method Distribution

Payment Method Share

Credit Card 75.7%

Boleto 19.9%

Voucher 2.87%

Debit Card 1.54%

Not Defined 0.00103%

Credit Card

Boleto

Voucher

Debit Card

Not Defined

Avg Order Value by Payment Type

Credit Card

Boleto

Voucher

Debit Card

Not Defined

Avg Order Value (R\$)

Payment Type

> Payment type detail table

Instalment Distribution (Credit Card Orders)

Only orders with payment_type = credit_card are included.

Instalment Count Distribution (Credit Card)

Orders

25k

20k

15k

10k

5k

0

1

2

3

4

5

6

7

8

9

10

11

12

13

Max Instalments

Average Order Value by Instalment Band (Credit Card)

Bands: 1 / 2-3 / 4-6 / 7-12 / 13+

AOV by Instalment Band

Avg Order Value (R\$)

350

300

250

200

150

100

50

0

1

2-3

4-6

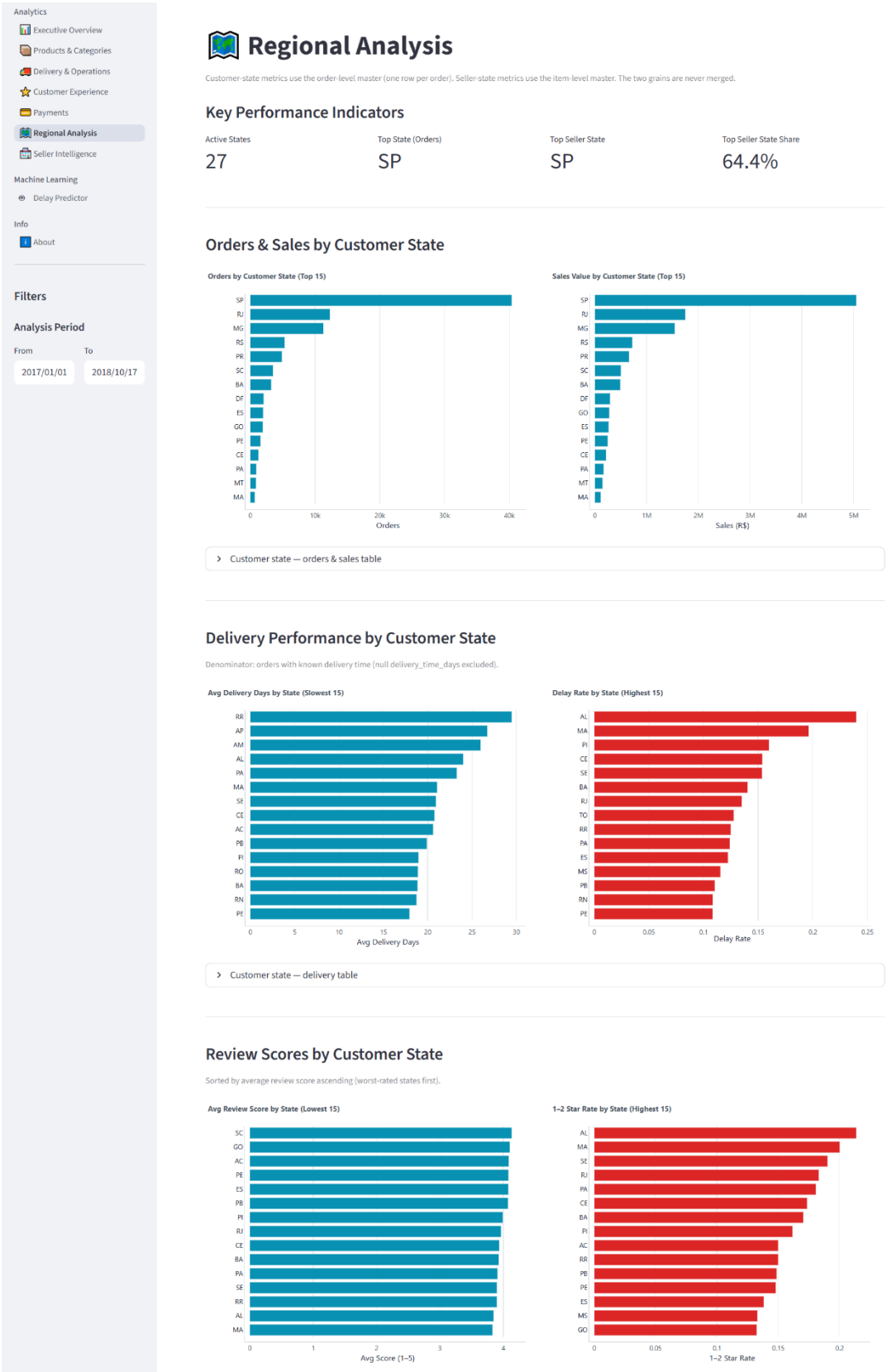
7-12

13+

Instalment Band

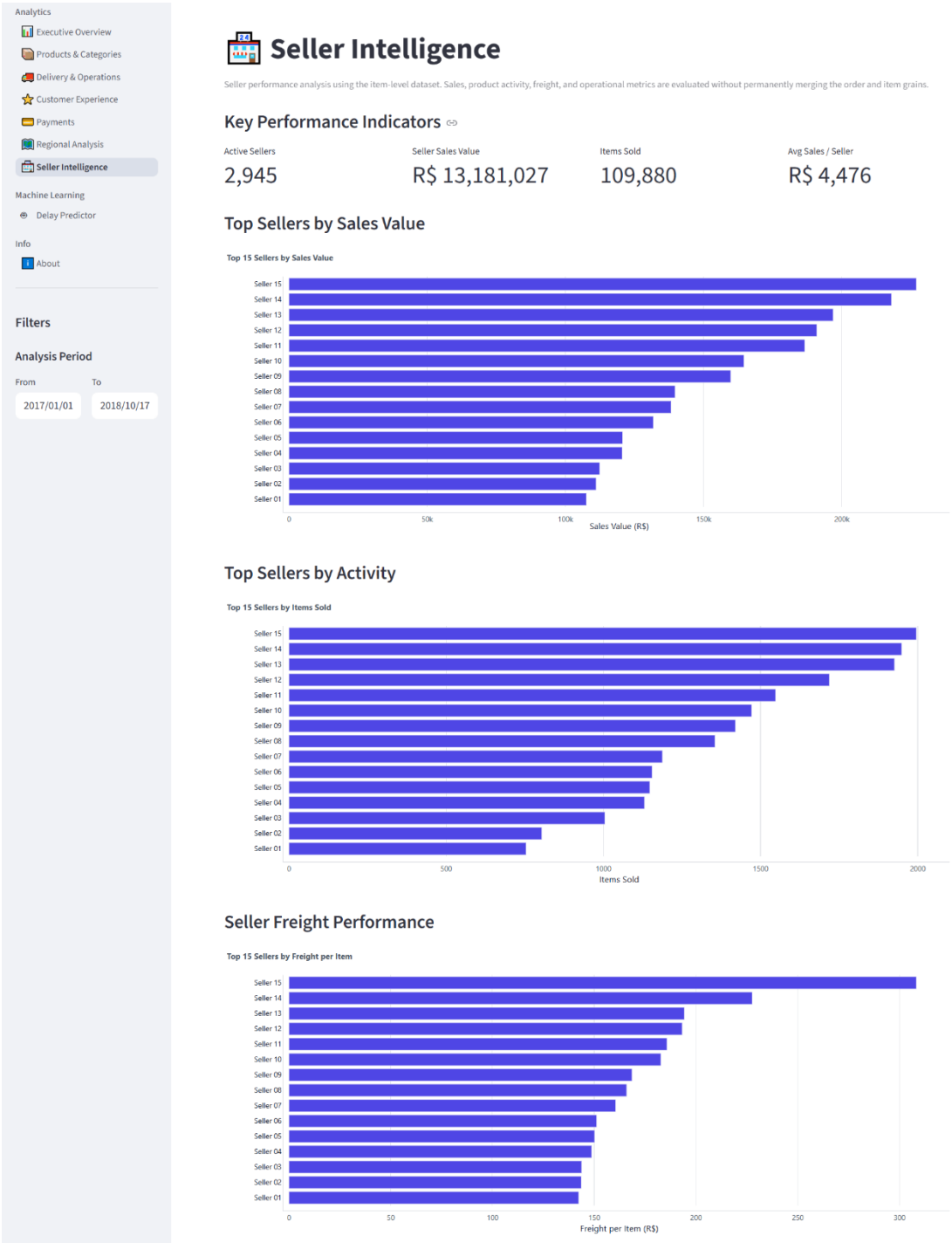
11.6 Section 6 — Regional Analysis

The Regional Analysis page examines customer locations, seller and customer regions, state-level performance, and regional order and sales activity.



11.7 Section 7 — Seller Intelligence

The Seller Intelligence page uses the item-level analytical grain to analyze seller sales value, items sold, seller activity levels, freight-related metrics, and rankings of top sellers by sales value and item activity.



11.8 Section 8 — Delay Predictor

The Delay Predictor page presents the machine-learning component of the platform, described in full in Section 13.

Analytics

Executive Overview

Products & Categories

Delivery & Operations

Customer Experience

Payments

Regional Analysis

Seller Intelligence

Machine Learning

Delay Predictor

Info

About

ML

Delivery Delay Predictor

Leakage-safe machine learning prediction for e-commerce delivery delays.

This model estimates the probability that an order will be delivered later than the estimated delivery date. It is an analytical prediction, not a guarantee.

Model Performance — Final Temporal Test

ROC-AUC

0.710

PR-AUC

0.104

Recall

75.1%

Precision

9.4%

Final test results from the untouched future-period test set. Prediction threshold = 0.50.

Order Prediction Inputs

Purchase Date

2018/06/15

Total Payment Value (R\$)

175.00

-

+

Product Weight (g)

1000.00

-

+

Purchase Time

14:00

▼

Number of Payment Methods

1

-

+

Product Length (cm)

20.00

-

+

Estimated Delivery Date

2018/06/25

Maximum Installments

1

-

+

Product Height (cm)

10.00

-

+

Number of Items

1

-

+

Primary Payment Type

credit_card

▼

Product Width (cm)

15.00

-

+

Total Product Price (R\$)

150.00

-

+

Product Category

bed_bath_table

▼

Customer State

SP

▼

Total Freight (R\$)

25.00

-

+

Seller State

SP

▼

Average Item Price (R\$)

150.00

-

+

Predict Delivery Delay

12. Business Questions Addressed

The platform is structured around eleven core business questions, each directly mapped to one or more dashboard modules in Section 11:

#	Business Question	Primary Module(s)
1	How are sales and orders performing over time?	Executive Overview
2	Which categories generate the highest sales value?	Products & Categories
3	Which categories have the highest item volumes?	Products & Categories
4	How do item prices and freight values vary?	Products & Categories
5	How does delivery performance vary?	Delivery & Operations
6	What proportion of orders experience delays?	Delivery & Operations
7	How does customer satisfaction vary?	Customer Experience
8	Which payment methods are most commonly used?	Payments
9	Which regions generate significant business activity?	Regional Analysis
10	How do sellers compare in sales and item activity?	Seller Intelligence
11	Can delivery-delay risk be estimated before the final outcome?	Delay Predictor (ML)

13. Machine Learning – Delay Predictor

13.1 Prediction Objective

The Delay Predictor estimates the likelihood that a given order will experience a delivery delay, defined as the actual delivery date falling after the estimated delivery date. The objective is to provide an early, probability-based signal of delivery risk rather than a certain outcome.

13.2 Target Variable

The target variable is `is_delayed`, a binary indicator defined only for orders with a resolved delivered status and a valid delivery date. Orders without a confirmed delivery outcome are excluded from the labeled modeling population rather than assigned an assumed value.

Category	Count
Labeled orders (valid delivered outcome)	96,470
Unlabeled orders (excluded from target)	2,971
Approximate delay rate (labeled population)	8.11%

The class imbalance implied by an ~8% positive rate is treated explicitly in modeling: evaluation relies on precision, recall, and ROC-AUC rather than accuracy alone, and class weighting is applied during training.

13.3 Training and Prediction Concept

The model is trained on the labeled population using features available prior to the delivery outcome, and is then used to generate a delay-probability estimate for new or existing orders based on those same pre-outcome features. This mirrors the standard predictive-analytics workflow of learning patterns from historical, labeled data and applying that learned relationship to estimate outcomes where the true result is not yet known.

13.4 Data Leakage Prevention

A core modeling principle applied throughout this project is the exclusion of post-outcome information from the feature set. Fields only known after an order has been delivered — including the actual delivery date and the customer review score (generated after the delivery experience) — are never used as predictive features. Including such fields would allow the model to indirectly learn the answer rather than genuinely predicting it, producing artificially strong but practically meaningless results.

Permitted feature categories:

- Order timing (purchase timestamp, day-of-week, month, processing-time-to-approval)
- Product attributes (category, weight, dimensions)
- Freight value and item price (known at purchase/shipment)
- Seller and customer geography (state-level distance proxy)
- Number of items in the order

Explicitly excluded (leakage) fields:

- order_delivered_customer_date
- delivery_time_days (derived from the actual delivery date)
- review_score and any review-derived field
- Any field only populated after delivery confirmation

13.5 Feature Engineering, Train/Test Split, and Model Training (scikit-learn)

```

"""
ml_module.py
Trains a leakage-safe Random Forest classifier to estimate
delivery-delay probability using only pre-outcome features.
"""

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score, classification_report

# 1. LOAD LEAKAGE-SAFE MODELING DATASET
ml_df = pd.read_csv("data/ml_delay_dataset.csv")
ml_df = ml_df.dropna(subset=["is_delayed"])

# 2. FEATURE ENGINEERING - pre-outcome features ONLY
ml_df["purchase_dayofweek"] = pd.to_datetime(ml_df["order_purchase_timestamp"]).dt.dayofweek
ml_df["purchase_month"] = pd.to_datetime(ml_df["order_purchase_timestamp"]).dt.month
ml_df["approval_lag_hours"] = (
    pd.to_datetime(ml_df["order_approved_at"])
    - pd.to_datetime(ml_df["order_purchase_timestamp"])
).dt.total_seconds() / 3600

NUMERIC_FEATURES = [
    "price", "freight_value", "product_weight_g",
    "product_length_cm", "product_height_cm", "product_width_cm",
    "item_count", "approval_lag_hours", "purchase_dayofweek", "purchase_month",
]
CATEGORICAL_FEATURES = [
    "product_category_name_english", "customer_state", "seller_state",
]

# Explicit leakage guard - fail loudly if a post-outcome field is present
LEAKAGE_FIELDS = {
    "order_delivered_customer_date", "delivery_time_days",
    "review_score", "delivery_delay_days",
}
assert LEAKAGE_FIELDS.isdisjoint(set(NUMERIC_FEATURES + CATEGORICAL_FEATURES)), \
    "Leakage field detected in feature set!"

X = ml_df[NUMERIC_FEATURES + CATEGORICAL_FEATURES]
y = ml_df["is_delayed"].astype(int)

# 3. TRAIN / TEST SPLIT - stratified on the imbalanced target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# 4. PREPROCESSING + MODEL PIPELINE

```

```

preprocessor = ColumnTransformer(
    transformers=[("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL_FEATURES)],
    remainder="passthrough",
)

model = RandomForestClassifier(
    n_estimators=300, max_depth=12,
    class_weight="balanced", # accounts for ~8% positive class rate
    random_state=42, n_jobs=-1,
)

pipeline = Pipeline(steps=[("preprocessor", preprocessor), ("classifier", model)])
pipeline.fit(X_train, y_train)

# 5. PROBABILITY PREDICTION AND EVALUATION
y_proba = pipeline.predict_proba(X_test)[: , 1] # probability of delay
y_pred = (y_proba >= 0.5).astype(int)

print("ROC-AUC:", roc_auc_score(y_test, y_proba))
print(classification_report(y_test, y_pred))

# 6. INFERENCE FUNCTION - used by pages/08_ml_delay_predictor.py
def predict_delay_probability(order_features: pd.DataFrame) -> np.ndarray:
    """
    Returns a delay-probability estimate (0-1) for each row in
    order_features. Input must contain only pre-outcome fields.
    """
    return pipeline.predict_proba(order_features)[: , 1]

```

13.6 Probability Output and Interpretation

The model produces a probability estimate representing the likelihood of delay for a given order, rather than a deterministic yes/no outcome. This probability is interpreted as a relative risk signal — orders with higher estimated probabilities warrant closer operational attention — but is never presented as a certainty.

13.7 Practical Business Use

The Delay Predictor is intended to support proactive operational decisions, such as flagging higher-risk orders for closer monitoring, informing customer-communication strategies, or supporting internal discussion about which sellers, regions, or categories are associated with elevated delay risk. It is a decision-support tool, not an automated decision-making system.

 Delivery Delay Predictor

Leakage-safe machine learning prediction for e-commerce delivery delays.

This model estimates the probability that an order will be delivered later than the estimated delivery date. It is an analytical prediction, not a guarantee.

Model Performance — Final Temporal Test

ROC-AUC	PR-AUC	Recall	Precision
0.710	0.104	75.1%	9.4%

Final test results from the untouched future-period test set. Prediction threshold = 0.50.

Order Prediction Inputs

Purchase Date	Total Payment Value (R\$)	Product Weight (g)
2018/06/15	175.00 - +	1000.00 - +
Purchase Time	Number of Payment Methods	Product Length (cm)
14:00 ▾	1 - +	20.00 - +
Estimated Delivery Date	Maximum Installments	Product Height (cm)
2018/06/25	1 - +	10.00 - +
Number of Items	Primary Payment Type	Product Width (cm)
1 - +	credit_card ▾	15.00 - +
Total Product Price (R\$)	Product Category	Customer State
150.00 - +	bed_bath_table ▾	SP ▾
Total Freight (R\$)		Seller State
25.00 - +		SP ▾
Average Item Price (R\$)		
150.00 - +		

 Predict Delivery Delay

Prediction Result

Predicted Delay Probability	Prediction
52.1%	Likely Delayed

The model predicts a delay probability at or above the 0.50 decision threshold.

The prediction is based only on information available at the order/purchase stage. Delivery outcome fields were excluded to prevent target leakage.

13.8 Limitations

The model's predictions are based on historical patterns within the Olist dataset's coverage period and do not account for factors outside the dataset, such as real-time logistics conditions, holidays, or external disruptions not reflected in the historical record.

14. Interactive Filtering

The platform implements a centralized filtering system, managed through `src/filters.py`, that applies consistently across all eight dashboard pages rather than allowing each page to define its own independent filtering logic.

Filter	Default Value
Date	2017-01-01 to 2018-10-17
Order Status	Delivered
Include 2016	OFF

Centralizing the filtering logic ensures that when a user adjusts the date range, order status, or 2016-inclusion setting, every dashboard page reflects the same underlying filtered population. This consistency is essential for a multi-page BI platform: without centralized filtering, different pages could silently present figures calculated over different underlying populations, undermining the reliability of cross-page comparisons and management-level interpretation.

```
"""
src/filters.py (excerpt)
Centralized filter state and application logic, shared by every page.
"""

import streamlit as st
import pandas as pd

def init_filter_state(default_start, default_end, default_status, include_2016):
    if "filters_initialized" not in st.session_state:
        st.session_state["date_start"] = pd.to_datetime(default_start)
        st.session_state["date_end"] = pd.to_datetime(default_end)
        st.session_state["order_status"] = default_status
        st.session_state["include_2016"] = include_2016
        st.session_state["filters_initialized"] = True

def apply_filters(df: pd.DataFrame, grain: str) -> pd.DataFrame:
    start = st.session_state["date_start"]
    end = st.session_state["date_end"]
    status = st.session_state["order_status"]

    filtered = df[
        (df["order_purchase_timestamp"] >= start)
        & (df["order_purchase_timestamp"] <= end)
    ]
    if status != "all":
        filtered = filtered[filtered["order_status"] == status]
    if not st.session_state["include_2016"]:
        filtered = filtered[filtered["order_purchase_timestamp"].dt.year != 2016]
    return filtered
```

15. Analytical Methodology

The platform's analysis is guided by nine principles, applied consistently across every module:

25. **Two-grain analytical architecture** — order-level and item-level questions are answered from the dataset whose grain matches the question (Section 7).
26. **Data quality and validation** — cleaning decisions and derived fields are checked against expected counts and distributions rather than assumed correct.
27. **Business-focused analytics** — every chart and KPI is tied to a defined business question rather than included for technical demonstration.
28. **Correct KPI aggregation** — KPIs are calculated at the appropriate grain to avoid inflation or dilution from mismatched joins.
29. **Avoiding duplicate counting** — order-level metrics are never calculated directly from item-level data without appropriate aggregation.
30. **No unsupported profit calculation** — the dataset does not include Cost of Goods Sold; price minus freight is never presented as profit.
31. **Correlation does not imply causation** — observed relationships are presented as associations to be investigated further, not as proven causal effects.
32. **Avoiding ML data leakage** — the Delay Predictor excludes any feature derived from post-outcome information (Section 13.4).
33. **Historical dataset limitations** — the dataset reflects a specific historical period and marketplace context; conclusions are scoped accordingly.

16. Validation and Testing

The project includes a dedicated suite of eleven automated validation scripts covering the data pipeline and every dashboard module:

- `_validate_app.py`
- `_validate_charts.py`
- `_validate_customer.py`
- `_validate_delivery.py`
- `_validate_filters.py`
- `_validate_kpis.py`
- `_validate_loader.py`
- `_validate_overview.py`
- `_validate_payments.py`
- `_validate_products.py`
- `_validate_regional.py`

Final validation results:

Validation Area	Result
Application	40 passed, 0 failed
Charts	64 passed, 0 failed
Customer Experience	69 passed, 0 failed
Delivery & Operations	59 passed, 0 failed
Filters	26 passed, 0 failed
KPIs	85 passed, 0 failed
Executive Overview	62 passed, 0 failed
Payments	55 passed, 0 failed
Products & Categories	48 passed, 0 failed
Regional Analysis	53 passed, 0 failed
Data Loader	All validations passed
Seller Intelligence	Validated separately and passed

Total automated assertions executed: 561, with zero failures reported across the entire suite.

In addition to the scripted validations above, the following checks were completed as part of final project sign-off:

- Python compilation completed successfully across the entire application codebase.
 - A final, clean Streamlit launch completed successfully with no runtime exceptions.
 - All eight dashboard pages loaded successfully during the final runtime check.
 - Live filter quality assurance was completed, confirming consistent behavior of the centralized filtering system across every page.
 - Edge-case quality assurance was completed (empty filter ranges, single-day ranges, all-status selection).
 - A final regression pass was completed successfully following the last round of code changes.
-

17. Project Structure

```
Olist-Ecommerce-Analytics/
|
+-- app.py
+-- README.md
+-- requirements.txt
|
+-- data/
|   +-- ml_delay_dataset.csv
|   +-- olist_master_cleaned.csv
|   +-- olist_order_item_master.csv
|
+-- pages/
|   +-- 01_overview.py
|   +-- 02_products_categories.py
|   +-- 03_delivery_operations.py
|   +-- 04_customer_experience.py
|   +-- 05_payments.py
|   +-- 06_regional.py
|   +-- 07_sellers.py
|   +-- 08_ml_delay_predictor.py
|
+-- src/
|   +-- data_loader.py
|   +-- filters.py
|   +-- kpis.py
|   +-- charts.py
|   +-- ml_module.py
|
+-- validation scripts
|   +-- _validate_app.py
|   +-- _validate_charts.py
|   +-- _validate_customer.py
|   +-- _validate_delivery.py
|   +-- _validate_filters.py
|   +-- _validate_kpis.py
|   +-- _validate_loader.py
|   +-- _validate_overview.py
|   +-- _validate_payments.py
|   +-- _validate_products.py
|   +-- _validate_regional.py
|
+-- GitHub configuration files
```

18. Installation and Execution

The application can be run locally by following the steps below.

1. Clone the repository

```
git clone https://github.com/brojomohan58-boop/Olist-Ecommerce-Analytics  
cd Olist-Ecommerce-Analytics
```

2. Create a virtual environment

```
python -m venv .venv
```

3. Activate the environment (Windows PowerShell)

```
.venv\Scripts\Activate.ps1
```

4. Install dependencies

```
pip install -r requirements.txt
```

5. Run the Streamlit application

```
streamlit run app.py
```

Once running, the application launches a local Streamlit server and opens the multi-page dashboard in the default web browser, beginning with the Executive Overview page.

19. GitHub Repository

The final project repository is hosted at:

<https://github.com/brojomohan58-boop/Olist-Ecommerce-Analytics>

The repository uses Git for version control and GitHub for remote hosting and submission reference. Because the analytical CSV datasets used by the application are relatively large, they are tracked using Git LFS (Large File Storage) rather than being committed directly to the standard Git history, keeping the repository efficient while preserving full access to the underlying data files.

20. Key Insights and Business Value

The platform is designed to surface insights across each of the business questions defined in Section 12. Where specific quantitative findings depend on live interaction with the filtering system and are best explored directly within the deployed dashboard, the analytical purpose of each area is summarized below:

- **Sales and order trends** allow the business to monitor growth or decline in activity over time and identify seasonal or period-specific patterns.
- **Category-level sales and volume analysis** helps distinguish between categories that generate high revenue and those that generate high transaction volume.
- **Delivery performance analysis** provides visibility into how reliably the operation meets its own estimated delivery commitments.
- **Customer experience analysis**, including the relationship between delivery outcomes and review scores, supports investigation into which operational factors may be associated with lower satisfaction.
- **Payment behavior analysis** informs understanding of how customers choose to transact, relevant to checkout design and promotional strategy.
- **Regional and seller analysis** identifies where business activity concentrates geographically and how sellers compare in scale of activity.
- **The Delay Predictor** adds a forward-looking dimension to the platform, allowing delivery risk to be estimated before an order's outcome is finalized.

Collectively, these components give the business a structured, validated view connecting sales performance, operational reliability, and customer satisfaction — the three areas most directly relevant to sustained marketplace performance.

21. Limitations

- Olist is a historical public marketplace dataset and does not represent the entire Brazilian e-commerce market.
- Historical patterns observed in this dataset should not automatically be interpreted as reflective of current market conditions.
- Seller IDs in the dataset do not provide seller business names; seller-level analysis is presented using identifiers only.
- The dataset does not include explicit Cost of Goods Sold (COGS) data.
- As a direct consequence, true profit cannot be calculated from the available fields, and no calculation in this project is presented as profit.
- Machine learning predictions produced by the Delay Predictor are probability estimates, not guarantees of future delivery outcomes.
- Observed relationships between variables in this platform are associative and should not be interpreted as proven causal relationships without further investigation.
- The scope of analysis is inherently limited by the fields and time period available in the dataset.

22. Future Improvements

The following represent realistic directions for future work and are not implemented in the current version of the platform:

- More advanced predictive modeling techniques for delivery-delay estimation (e.g., gradient boosting ensembles, hyperparameter search).
- Model monitoring and periodic retraining to maintain predictive relevance over time.

- Integration of additional external business data sources to expand analytical scope.
- More granular geographic analysis beyond state-level aggregation.
- Improved seller metadata, should richer seller information become available.
- Automated deployment of the dashboard to a hosted environment.
- Additional forecasting capabilities for sales and demand.
- More advanced customer segmentation analysis.
- More comprehensive experimentation and causal-analysis methods where appropriate data and design permit.

These items are identified as future work only and are not represented as part of the current, submitted platform.

23. Project Outcome

This project successfully demonstrates a complete, end-to-end analytics workflow: raw relational e-commerce data was cleaned and transformed into two purpose-built analytical datasets; business-relevant KPIs were designed and validated; exploratory data analysis was conducted across sales, delivery, and customer-experience domains; a leakage-controlled machine learning model was developed to estimate delivery-delay risk; the resulting analysis was made accessible through an interactive, multi-page Streamlit application; and the entire pipeline and application were validated through a structured suite of automated tests.

The project demonstrates practical proficiency in data preparation, data modeling, KPI design, exploratory analysis, business intelligence dashboard development, applied machine learning, and software validation and testing practices. It represents a functional, internally consistent analytics platform appropriate to the scope of an internship capstone.

24. Conclusion

The E-Commerce Business Intelligence & Customer Experience Analytics Platform fulfills the objectives set out at the start of this project: transforming raw, multi-table marketplace data into a validated, interactive analytics platform capable of supporting real business questions around sales performance, delivery reliability, customer experience, and delivery-risk estimation. Throughout the project, analytical claims were deliberately scoped to what the underlying data could support — profitability was not claimed where cost data did not exist, causal relationships were not claimed where only associations could be observed, and machine-learning outputs were presented as probability estimates rather than certainties. This disciplined approach, combined with structured validation testing across every application module, produced a reliable and defensible analytics platform suitable for submission as the culminating project of the AICTE | IBM SkillsBuild Data Analytics with AI Internship 2026.

25. Author Information

Field	Detail
Author	Brojo Mohan Dutta
Education	B.Sc. (Honours) Mathematics, Serampore College, University of Calcutta, 2025

Field	Detail
Project Context	AICTE IBM SkillsBuild Data Analytics with AI Internship 2026
Repository	https://github.com/brojomohan58-boop/Olist-Ecommerce-Analytics